INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	SW-AyLP-APE-03: Estructuras de Repetición.
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Primero Software "B"
Alumnos participantes:	Bravo López Jordán Samuel Sánchez Lema Isaac Adrian ……..
Asignatura:	ALGORITMOS Y LÓGICA DE PROGRAMACIÓN
Docente:	Ing. José Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

1.1 Objetivos

General:
Utilizar estructuras de repetición en el análisis y diseño de algoritmos.

Específicos:

- Implementar una serie de bucles (como for o while) para manipular una lista compuesta por varios estudiantes de diferentes cursos y paralelos, permitiendo el cálculo intensivo de promedios sin necesidad de ejecutar manualmente cada uno de los procesos necesarios que compone un estudiante.
- Emplear estructuras de repetición con el propósito de validar la entrada de datos pertenecientes a un sistema dentro de un rango determinado (ejemplo, notas del rango 0 a 10), hasta lograr una armonía en el sistema, generalizando el control además del orden.
- Aplicar las diferentes estructuras cíclicas para actualizar dinámicamente las variables tales como la suma total, conteo de aprobados/reprobados a medida que se procesa cada registro académico en relación a cada uno de los estudiantes presentes al momento de ingresar sus datos.

1.2 Modalidad
Presencial.

1.3 Tiempo de duración
Presenciales: 6.
No presenciales: 0.

1.4 Instrucciones
- El trabajo se desarrollará de manera individual.
- Lea detenidamente cada uno de los ejercicios planteados en el aula virtual y realice la codificación en C++.
- Subir en un documento en formato PDF que contenga capturas del código de cada ejercicio, adicionalmente, comprimir en una carpeta los archivos con el código generado, y subirlo al aula virtual.
- Si se manifiestan dudas se invita a los estudiantes a acudir a las tutorías académicas.

1.5 Listado de equipos, materiales y recursos

- **Inteligencia artificial, TAC Inteligencia artificial, TAC Inteligencia artificial, TAC.**
- **- Computador o laptop.**
- **- Plataforma educativa de la UTA.**

- **- Internet para consultas. - Papel - Material de escritorio.**

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

☒Plataformas educativas
☒Simuladores y laboratorios virtuales
☐Aplicaciones educativas
☒Recursos audiovisuales
☐Gamificación
☐Inteligencia Artificial
Otros (Especifique): _____

1.6 Actividades por desarrollar

Desarrollar la guía APE en base al formato y las instrucciones brindadas.

1.7 Resultados obtenidos

Procesamiento Eficiente de Datos

Se logró la capacidad de procesar un número indeterminado de registros de estudiantes en una sola ejecución. Gracias al uso de estructuras como while o do-while, el sistema no se detiene tras calcular un solo promedio, sino que permite el ingreso continuo de datos hasta que el usuario decide finalizar el proceso.

Validación Robusta de Entradas

El algoritmo eliminó el margen de error por ingresos accidentales. Se verificó que el sistema rechaza cualquier calificación fuera del rango (ej. menores a 0 o mayores a 10), obligando al usuario a repetir el ingreso mediante un bucle de validación. Esto asegura que los cálculos finales (promedios y estados de aprobación) sean **100% confiables**.

Generación de Estadísticas Académicas

A través del uso de contadores y acumuladores dentro de las repeticiones, el programa entrega al finalizar la ejecución un resumen preciso que incluye:

- El **promedio general** de todo el curso.
- El **número total** de estudiantes aprobados y reprobados.
- El porcentaje de éxito académico del grupo analizado.

1.8 Habilidades blandas empleadas en la práctica

☒ Liderazgo
☒ Trabajo en equipo
☐ Comunicación asertiva
☒ La empatía
☐ Pensamiento crítico
☐ Flexibilidad
☐ La resolución de conflictos
☐ Adaptabilidad
☒ Responsabilidad

1.9 Conclusiones

Se determinó que el uso de las diferentes sentencias if-else además de switch es esencial para desarrollar programas participativos y fuertes. En el ejercicio del sistema bancario, estas estructuras permitieron validar condiciones críticas (como la existencia de saldo suficiente) antes de ejecutar una operación, garantizando que el flujo del algoritmo responda lógicamente a las entradas del usuario y evite estados inconsistentes en los datos.

La implementación de la clase Jean demostró que la POO facilita el mantenimiento y la escalabilidad del código. Al encapsular los datos (como la humedad y el color) y protegerlos bajo métodos específicos (lavar, secar), se logró una representación fiel de los procesos físicos de una fábrica o tienda, asegurando que los atributos solo se modifiquen mediante comportamientos predefinidos y no por accesos externos no autorizados.

A través del desarrollo del control de hora y los menús cíclicos (do-while), se concluye que la combinación de estructuras repetitivas con selectivas mejoras significativamente la experiencia del usuario. Esta metodología permite que el software sea flexible, dándole al usuario el control total sobre la manipulación de la información en tiempo real (modificar datos registrados o realizar múltiples transacciones) sin necesidad de reiniciar la ejecución del programa.

1.10 Recomendaciones

Para futuros proyectos con estructuras de datos más complejas (como arreglos bidimensionales o matrices), se recomienda el uso de bucles anidados. Esto permitiría, por ejemplo, gestionar múltiples asignaturas para cada estudiante dentro del mismo ciclo de procesamiento.

Aunque los bucles validan rangos numéricos, se sugiere integrar bloques try-catch o validaciones de tipo de dato. Esto evitaría que el programa se cierre inesperadamente si el usuario ingresa texto (letras) en lugar de números durante la repetición.

Se recomienda extraer la lógica de los bucles hacia funciones o métodos independientes. Esto mejora la legibilidad del código y permite que la estructura de repetición sea más limpia, invocando una función de "Cálculo de Promedio" en cada iteración.

1.11 Referencias bibliográficas

[1] I. Isma1402uta-boop, "Implementación de Sistema de Análisis de Rendimiento Académico / SW-AyLP-APE-03: Estructuras de Repetición," *GitHub*, Issue #1, abr. 01, 2026. [En línea]. Disponible en: https://github.com/isma1402uta-boop/Sistema-Academico-SW-AyLP-APE-03-Estructuras-de-Repetici-n/issues/1. [Accedido: abr. 01, 2026].

EJERCICIO

Sistema de análisis de rendimiento académico con control estricto de datos y reportes iterativos:

Una carrera universitaria desea analizar el rendimiento de un grupo de estudiantes en una asignatura. Se debe desarrollar un programa en C++ que permita ingresar información académica de N estudiantes, donde N debe validarse para que sea mayor que 0.

Por cada estudiante se debe registrar:

- nombre del estudiante,
- paralelo,
- nota del primer parcial,
- nota del segundo parcial,
- nota de prácticas,
- porcentaje de asistencia.
- El sistema debe validar que:
- las notas estén entre 0 y 10,
- la asistencia esté entre 0 y 100,
- el paralelo solo pueda ser A, B o C.
- Para cada estudiante, el programa debe calcular:
- promedio final ponderado, donde:
- primer parcial = 30%
- segundo parcial = 30%
- prácticas = 40%
- Luego clasificar al estudiante en una categoría:
- Excelente: promedio >= 9
- Muy bueno: promedio >= 8 y < 9
- Bueno: promedio >= 7 y < 8
- En riesgo: promedio >= 6 y < 7
- Reprobado: promedio < 6
- Pero además:
- Si la asistencia es menor al 70%, el estado final debe ser “Reprobado por asistencia”, aunque el promedio sea alto.
- Si un estudiante tiene promedio >= 9 pero asistencia menor a 80%, debe generarse una alerta académica por inconsistencia.
- Si las tres notas son exactamente iguales, debe marcarse como rendimiento estable.
- Si la diferencia entre la nota mayor y la menor supera 4 puntos, debe marcarse como rendimiento inestable.
- Al finalizar el ingreso de todos los estudiantes, el sistema debe mostrar:
- Reporte general
- Total de estudiantes procesados.
- Cantidad de estudiantes por paralelo.
- Cantidad de estudiantes en cada categoría.
- Cantidad de reprobados por nota.
- Cantidad de reprobados por asistencia.
- Promedio general del curso.
- Promedio general por paralelo.
- Mayor promedio del curso y nombre del estudiante.

- Menor promedio del curso y nombre del estudiante.
- Porcentaje de estudiantes aprobados.
- Porcentaje de estudiantes con rendimiento estable.
- Porcentaje de estudiantes con rendimiento inestable.
- Reporte adicional
- El programa debe preguntar al final si el usuario desea repetir todo el proceso para otro curso. Para ello debe utilizar una estructura repetitiva adecuada.

1) **Análisis del problema**

El sistema requiere procesar el rendimiento académico de una cantidad variable de estudiantes. El desafío principal consiste en integrar tres variables críticas: **calificaciones cuantitativas**, **cumplimiento de asistencia** y **estabilidad del aprendizaje** (variabilidad de notas), para emitir un diagnóstico individual y un reporte estadístico global por grupos (Paralelos A, B y C).

Lógica de Procesamiento (Cálculos y Reglas)

El núcleo del análisis se basa en las siguientes fórmulas y condiciones lógicas:

- **Cálculo del Promedio Ponderado:**

  Promedio = (N1 * 0.30) + (N2 *0.30) + (Practicas *0.40)

- **Análisis de Rendimiento:**
  - **Estable:** Si las tres notas son idénticas (N1 = N2 = Practicas).
  - **Inestable:** Si la brecha entre la nota más alta y la más baja es > 4 puntos.
  - **Normal:** Casos intermedios.

2) **Variables de entrada, proceso y salida**

Tabla 1. Variables del ejercicio

Categoría	Variable	Tipo de Dato
Entrada	**Total**	**Int**
	NombreEstuiante	**string**
	Paralelo	**Char**
	N1	**Float**
	N2	**Float**
	Practicas	**Float**
	Asistencia	**Float**
Proceso	**Promedio**	**Float**
	Sumatotal	**Float**
	Sumapromedio	**Float**
	Mejorpromedio	**Float**
	Peorpromedio	**Float**
	Notamx	**Float**
	ContadorCurso	**Int**
	contadorAprobados	**Int**
	Reprovadosnta	**Int**
	Contadoresxcelente	**Int**
Salida	**Estado**	**String**
	Categoría	**String**
	Rendimiento	**String**
	Notamejor	**String**
	Notapeor	**String**

3) **Algoritmos (pSeInt)**

Inicio del Proceso Principal

1. **Inicialización:** Preparar un ciclo para permitir procesar múltiples cursos si el usuario lo desea.
2. **Definición de Alcance:** Solicitar al usuario la cantidad total de estudiantes a evaluar.
3. **Configuración de Contadores:** Poner en cero todos los acumuladores (sumas de promedios) y contadores (aprobados, reprobados, tipos de rendimiento).
4. **Bucle de Captura (Para cada estudiante):**
    - **Entrada de Datos:** Solicitar nombre, paralelo (A, B o C), dos notas parciales, nota de prácticas y porcentaje de asistencia.
    - **Validación:** Asegurar que las notas estén entre 0 y 10, y la asistencia entre 0 y 100.
    - **Cálculo de Promedio**
    - **Análisis de Estabilidad:**
        - Si todas las notas son iguales, marcar como "Estable".
        - Si la diferencia entre la nota más alta y la más baja es mayor a 4 puntos, marcar como "Inestable".
        - De lo contrario, marcar como "Normal".
    - **Registro de Extremos:** Comparar el promedio actual con el mejor y peor promedio registrados hasta ahora para actualizar los nombres de los estudiantes líderes y rezagados.
    - **Acumulación Estadística:** Sumar el promedio al total del curso y al total específico de su paralelo (A, B o C).

5. **Generación de Resultados:**
   - Mostrar la tabla detallada con los resultados individuales.
   - Calcular y mostrar el promedio general del grupo.
   - Calcular y mostrar los promedios independientes por cada paralelo.
   - Presentar los porcentajes de aprobación y de rendimiento estable/inestable.
6. **Cierre:** Preguntar al usuario si desea reiniciar el proceso para un nuevo grupo. Si la respuesta es negativa, finalizar el programa.

Fin del Algoritmo

4) Pseudocódigo (pSeInt)

```
Algoritmo Gestion_Academica_Estudiantes

   // 1. Declaracion de Variables

   Definir total, i Como Entero

   Definir cA, cB, cC, cExc, cMB, cBno, cRiesgo, cRepNota, cRepAsis, cAprob, cEst,
cInest Como Entero

   Definir n1, n2, prac, asis, prom, sumaT, sumaA, sumaB, sumaC, mejorP, peorP,
nMax, nMin Como Real

   Definir nombre, paralelo, estado, categoria, rendimiento, mejorEst, peorEst, repeti
Como Cadena


         Repetir

               // 2. Inicializacion de contadores y acumuladores

               cA<-0; cB<-0; cC<-0; cExc<-0; cMB<-0; cBno<-0; cRiesgo<-0

               cRepNota<-0; cRepAsis<-0; cAprob<-0; cEst<-0; cInest<-0

               sumaT<-0; sumaA<-0; sumaB<-0; sumaC<-0; mejorP<--1; peorP<-
11


               Escribir "Ingrese la cantidad de estudiantes a procesar:"

               Leer total


               Para i <- 1 Hasta total Hacer

                     Escribir "--- DATOS DEL ESTUDIANTE #", i, " ---"


                     Escribir "Nombre Completo:"
```

```
Leer nombre
Escribir "Paralelo (A/B/C):"
Leer paralelo
paralelo <- Mayusculas(paralelo)

Escribir "Nota Parcial 1 (0-10):"
Leer n1
Escribir "Nota Parcial 2 (0-10):"
Leer n2
Escribir "Nota Practicas (0-10):"
Leer prac
Escribir "Porcentaje de Asistencia (0-100):"
Leer asis

// 3. Calculos Academicos
prom <- (n1 * 0.30) + (n2 * 0.30) + (prac * 0.40)
sumaT <- sumaT + prom

// Clasificacion por nota (Categoria)
Si prom >= 9 Entonces
    categoria <- "Excelente"
    cExc <- cExc + 1
Sino
    Si prom >= 8 Entonces
        categoria <- "Muy Bueno"
        cMB <- cMB + 1
    Sino
        Si prom >= 7 Entonces
```

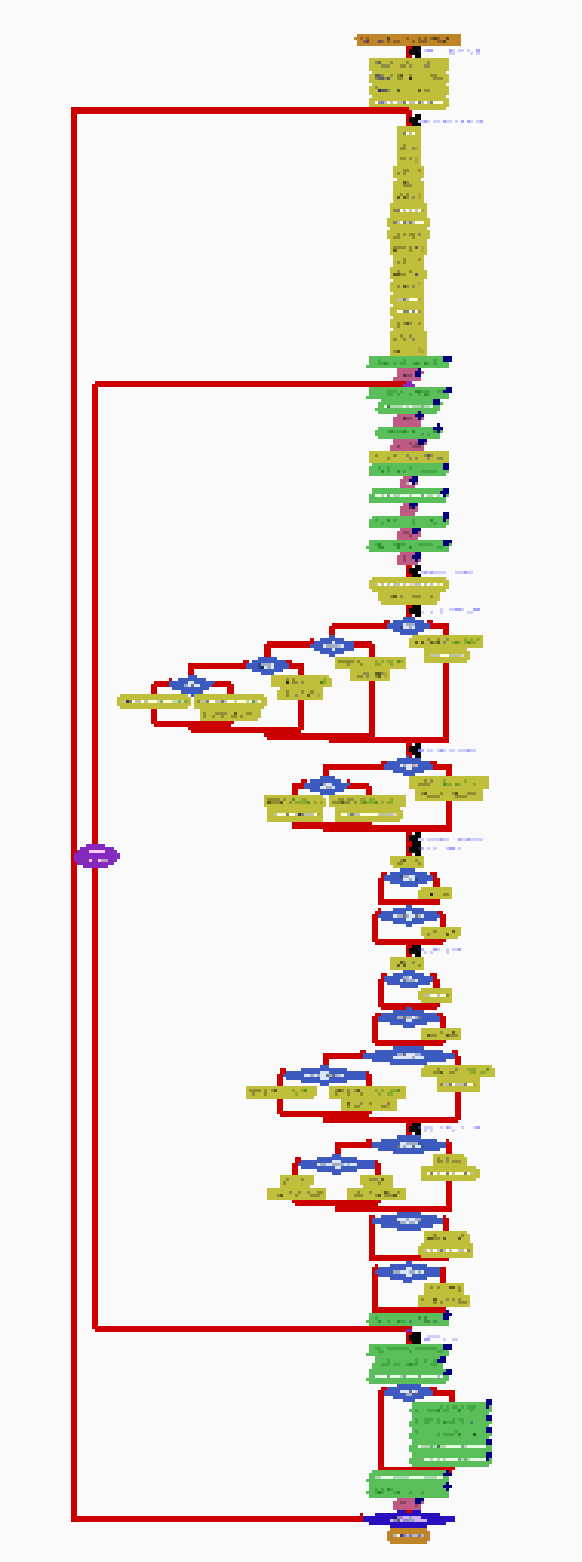
```
                    categoria <- "Bueno"
                    cBno <- cBno + 1
            Sino
                    Si prom >= 6 Entonces
                            categoria <- "En Riesgo"
                            cRiesgo <- cRiesgo + 1
                    Sino
                            categoria <- "Reprobado"
                    FinSi
            FinSi
    FinSi
FinSi

// Estado por Asistencia
Si asis < 70 Entonces
    estado <- "Reprobado por asistencia"
    cRepAsis <- cRepAsis + 1
Sino
    Si prom < 6 Entonces
        estado <- "Reprobado por nota"
        cRepNota <- cRepNota + 1
    Sino
        estado <- "Aprobado"
        cAprob <- cAprob + 1
    FinSi
FinSi

// 4. Analisis de Rendimiento (Estabilidad)
```

```
        // Encontrar Maximo
        nMax <- n1
        Si n2 > nMax Entonces nMax <- n2
        FinSi

Si prac > nMax Entonces nMax <- prac
FinSi

        // Encontrar Minimo
        nMin <- n1
        Si n2 < nMin Entonces nMin <- n2
        FinSi

Si prac < nMin Entonces nMin <- prac
FinSi

        Si (n1 = n2 Y n2 = prac) Entonces
                rendimiento <- "Estable"
                cEst <- cEst + 1
        Sino
                Si (nMax - nMin) > 4 Entonces
                        rendimiento <- "Inestable"
                        cInest <- cInest + 1
                Sino
                        rendimiento <- "Normal"
                FinSi
        FinSi
```

```
// 5. Estadisticas por Paralelo y Extremos
Si paralelo = "A" Entonces
    cA <- cA + 1
    sumaA <- sumaA + prom
Sino
    Si paralelo = "B" Entonces
        cB <- cB + 1
        sumaB <- sumaB + prom
    Sino
        cC <- cC + 1
        sumaC <- sumaC + prom
    FinSi
FinSi

Si prom > mejorP Entonces
    mejorP <- prom
    mejorEst <- nombre
FinSi

Si prom < peorP Entonces
    peorP <- prom
    peorEst <- nombre
FinSi

Escribir "Resultado: ", nombre, " - Promedio: ", prom, " - Estado: ", estado
Fin Para

// 6. Reporte Final
```

```
        Escribir "==========================================="
        Escribir "RESUMEN ESTADISTICO"
        Escribir "Total Procesados: ", total
        Si total > 0 Entonces
            Escribir "Promedio General del Curso: ", sumaT / total
            Escribir "Mejor Estudiante: ", mejorEst, " (", mejorP, ")"
            Escribir "Peor Estudiante: ", peorEst, " (", peorP, ")"
            Escribir "Porcentaje de Aprobados: ", (cAprob / total) * 100,
"%"
            Escribir "Rendimiento Estable: ", (cEst / total) * 100, "%"
        FinSi
        Escribir "==========================================="

        Escribir "¿Desea procesar otro curso? (S/N):"
        Leer repeti
        Hasta Que(repeti = "N" O repeti = "n")

FinAlgoritmo
```

5) Diagrama de flujo (pSeInt)



6) Código en C++ (codeblocs)

```
#include <iostream>

#include <string>

#include <limits>

#include <iomanip>

#include <algorithm>

#include <cctype>

#include <stdexcept>


using namespace std;


struct Estudiante {

    string nombreEstudiante;

    char paralelo;

    float n1, n2, practicas, promedio, asistencias;

    string estado;

    string categoria;

    string rendimiento;

};


string LeerNombre(string mensaje){

    string nombreEstudiante;


    while(true){

        cout<<mensaje;

        getline(cin>> ws, nombreEstudiante);


        bool valido = true;
```

```
        for(char c : nombreEstudiante){
            if(!isalpha(c) && !isspace(c)){
                valido = false;
                break;
            }
        }

        if(nombreEstudiante.empty()){
            cout<<"Error: El nombre del estudiante se encuentra vacio."<<endl;
        }else if(!valido){
            cout<<"Error: El nombre solo debe contener letras y espacios."<<endl;
        }else{
            return nombreEstudiante;
        }
    }
}

char LeerParalelo(string mensaje){
    char paralelo;

    while(true){
        cout<<mensaje;
        cin>> paralelo;
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        paralelo = toupper(paralelo);

        if(paralelo == 'A' || paralelo == 'B' || paralelo == 'C'){
```

```
            return paralelo;
        }else{
            cout << "Error: El paralelo solo puede ser A, B o C." << endl;
        }
    }
}

float LeerNotas(string mensaje, float min, float max){
    float nota;
    string texto;
    while(true){
        cout<<mensaje;
        getline(cin>> ws, texto);

        if(texto.empty()){
            cout<<"Error: El campo se encuentra vacio, Ingrese una nota."<<endl;
            continue;
        }

        try{
            nota = stof(texto);
            if(nota > max || nota < min){
              cout<<"Error: Ingrese una nota dentro del rango de "<<min<<" a "<<max<<"
."<<endl;
               continue;
            }
            return nota;

        }catch(invalid_argument& e){
```

```
            cout<<"Error: Ingrese una nota real."<<endl;
        }catch(out_of_range& e){
            cout<<"Error: El numero es demasiado grande."<<endl;
        }catch(...){
            cout<<"Error: Ingrese olo numeros."<<endl;
        }
    }
}

int LeerEstudiantes(string mensaje, int min){
    string texto;
    int n;

    while(true){
        cout<< mensaje;
        getline(cin >> ws, texto);
        if(texto.empty()){
            cout<<"Error.Llene el campo solicitado."<<endl;
            continue;
        }
        try{
            n = stoi(texto);
            if(n < min){
                cout << "Error: La cantidad debe ser al menos " << min << "." << endl;
                continue;
            }
            return n;
```

```
        }catch(invalid_argument& e){
            cout<<"Error.Ingrese solo numeros ."<<endl;
        }catch(out_of_range& e){
            cout<<"Error.El numero es muy grande."<<endl;
        }catch(...){
            cout<<"Error.Solo numeros."<<endl;
        }
    }
}

int main() {

    char repetir;

    do{
        system("cls");

            //Numero de estudiantes
            int total = LeerEstudiantes("Ingrese la cantidad de estudiantes a procesar: ", 1);



            // Contadores de categorias y estados


            // Estudiantes por paralelo
            int contadorCursoA = 0, contadorCursoB = 0, contadorCursoC = 0;


            // Categorias por calificacion del estudiante
            int contadorExcelente = 0, contadorMuyBueno = 0, contadorBueno = 0
```

```
, contadorRiesgo = 0, cReprobadoNota = 0;

//Contadores de estado y rendimiento

// Estados
int contadorReprobadosAsistencia = 0, contadorAprobados = 0;

int contadorNotasEstable = 0, contadorNotasInestable = 0;

// Acumuladores para promedios
float sumaTotal = 0, sumaPromedioA = 0, sumaPromedioB = 0, sumaPromedioC
= 0;

// Para el mejor y peor estudiante
float mejorPromedio = -1, peorPromedio = 11;
string notaMejor, notaPeor;

Estudiante* lista = new Estudiante[total];

for (int i = 0; i < total; i++) {

    //Sistema de datos
    cout << "\n----------------------------------------";
    cout << "\nDATOS DEL ESTUDIANTE #" << i + 1 << endl;
```

```
lista[i].nombreEstudiante = LeerNombre("Nombre del Estudiante (Completo): ");
lista[i].paralelo = LeerParalelo("Paralelo del Estudiante (A/B/C): ");
lista[i].n1 = LeerNotas("Nota del Primer Parcial : ", 0, 10);
lista[i].n2 = LeerNotas("Nota del Segundo Parcial : ", 0, 10);
lista[i].practicas = LeerNotas("Nota de Practicas del Estudiante: ", 0, 10);
lista[i].asistencias = LeerNotas("Porcentaje de Asistencia (0-100): ", 0, 100);

//Promedio Final Ponderado
lista[i].promedio = (lista[i].n1 * 0.30) + (lista[i].n2 * 0.30) + (lista[i].practicas * 0.40);

//Clasificacion del estudiante
if (lista[i].promedio >= 9) lista[i].categoria = "Excelente";
else if (lista[i].promedio >= 8) lista[i].categoria = "Muy bueno";
else if (lista[i].promedio >= 7) lista[i].categoria = "Bueno";
else if (lista[i].promedio >= 6) lista[i].categoria = "En riesgo";
else lista[i].categoria = "Reprobado";

//Sistema de asistencia
if (lista[i].asistencias < 70) {
    lista[i].estado = "Reprobado por asistencia";
} else {
    lista[i].estado = "Aprobado";
    if (lista[i].promedio >= 9 && lista[i].asistencias < 80) {
        lista[i].estado = "ALERTA ACADEMICA (Inconsistencia)";
    }
```

```
}



//Sistema de Rendimiento
float notaMax = max({lista[i].n1, lista[i].n2, lista[i].practicas});
float notaMin = min({lista[i].n1, lista[i].n2, lista[i].practicas});

if (lista[i].n1 == lista[i].n2 && lista[i].n2 == lista[i].practicas) {
   lista[i].rendimiento = "Estable";
}

else if ((notaMax - notaMin) > 4) {
   lista[i].rendimiento = "Inestable";
}
else {
   lista[i].rendimiento = "Normal";
}

// 1. Acumuladores
sumaTotal += lista[i].promedio;

// 2. Por Paralelo
if (lista[i].paralelo == 'A') {
   contadorCursoA++;
   sumaPromedioA += lista[i].promedio;

} else if (lista[i].paralelo == 'B') {
   contadorCursoB++;
```

```
    sumaPromedioB += lista[i].promedio;

} else if (lista[i].paralelo == 'C') {
    contadorCursoC++;
    sumaPromedioC += lista[i].promedio;
}

// 3 y 4. Categoría y Estado
if (lista[i].categoria == "Excelente") contadorExcelente++;
else if (lista[i].categoria == "Muy bueno") contadorMuyBueno++;
else if (lista[i].categoria == "Bueno") contadorBueno++;
else if (lista[i].categoria == "En riesgo") contadorRiesgo++;

// Manejo preciso de reprobados y aprobados
if (lista[i].estado == "Reprobado por asistencia") {
    contadorReprobadosAsistencia++;

} else if (lista[i].promedio < 6) { // Regla de la imagen anterior
    cReprobadoNota++;

} else {
    contadorAprobados++;
}

// 5. Rendimiento
if (lista[i].rendimiento == "Estable") contadorNotasEstable++;
else if (lista[i].rendimiento == "Inestable") contadorNotasInestable++;
```

```
    // 6. Extremos (Mejor y Peor)
    if (lista[i].promedio > mejorPromedio) {
        mejorPromedio = lista[i].promedio;
        notaMejor = lista[i].nombreEstudiante;

    }
    if (lista[i].promedio < peorPromedio) {
        peorPromedio = lista[i].promedio;
        notaPeor = lista[i].nombreEstudiante;

    }

}

//1. Encabezadp
cout << "\n" << setfill('=') << setw(105) << "=" << endl;
cout << setfill(' ') << left
     << setw(35) << "Nombre"
     << setw(10) << "Prom"
     << setw(12) << "Asis (%)"
     << setw(28) << "Estado"
     << "Rendimiento" << endl;
cout << setfill('-') << setw(105) << "-" << setfill(' ') << endl;

// 2. Cuerpo de la tabla
for (int i = 0; i < total; i++) {

    string asisConSigno = to_string((int)lista[i].asistencias) + "%";
```

```
    cout << left
        << setw(35) << lista[i].nombreEstudiante
        << setw(10) << fixed << setprecision(2) << lista[i].promedio
        << setw(12) << asisConSigno
        << setw(28) << lista[i].estado
        << lista[i].rendimiento << endl;
    }
    cout << setfill('=') << setw(105) << "=" << setfill(' ') << endl;

    // --- REPORTE GENERAL FINAL ---
    cout << fixed << setprecision(2);
    cout << "\n" << setfill('') << setw(65) << "" << endl;
    cout << setfill(' ') << "                RESUMEN ESTADISTICO DEL CURSO" << endl;
    cout << setfill('') << setw(65) << "" << setfill(' ') << endl;

    // 1, 2 y 3. Conteos básicos
    cout << "1. Total de estudiantes procesados: " << total << endl;
    cout << "2. Estudiantes por paralelo: [A: " << contadorCursoA
        << "] [B: " << contadorCursoB << "] [C: " << contadorCursoC << "]" << endl;
    cout << "3. Categorias: [Exc: " << contadorExcelente << "] [MB: " << contadorMuyBueno
        << "] [B: " << contadorBueno << "] [Riesgo: " << contadorRiesgo << "]" << endl;

    // 4 y 5. Reprobados
    cout << "4. Cantidad de reprobados por nota: " << cReprobadoNota << endl;
```

```
        cout << "5. Cantidad de reprobados por asistencia: " << contadorReprobadosAsistencia << endl;

    // 6 y 7. Promedios con validación
    cout << "6. Promedio general del curso: " << (total > 0 ? sumaTotal / total : 0.0) << endl;
    cout << "7. Promedio por paralelo: " << endl;
        cout << "   - Paralelo A: " << (contadorCursoA > 0 ? sumaPromedioA / contadorCursoA : 0.0) << endl;
        cout << "   - Paralelo B: " << (contadorCursoB > 0 ? sumaPromedioB / contadorCursoB : 0.0) << endl;
        cout << "   - Paralelo C: " << (contadorCursoC > 0 ? sumaPromedioC / contadorCursoC : 0.0) << endl;

    // 8 y 9. Extremos
        cout << "8. Mayor promedio: " << mejorPromedio << " (Estudiante: " << notaMejor << ")" << endl;
        cout << "9. Menor promedio: " << peorPromedio << " (Estudiante: " << notaPeor << ")" << endl;

    // 10, 11 y 12. Porcentajes
    if (total > 0) {
        cout << "10. Porcentaje de aprobados: " << (float)contadorAprobados / total * 100.0 << "%" << endl;
        cout << "11. Porcentaje rendimiento estable: " << (float)contadorNotasEstable / total * 100.0 << "%" << endl;
        cout << "12. Porcentaje rendimiento inestable: " << (float)contadorNotasInestable / total * 100.0 << "%" << endl;
    }

    cout << setfill('=') << setw(65) << "=" << setfill(' ') << endl;
```

```
    delete[] lista;

    cout << "\n" << setfill('-') << setw(65) << "-" << endl;
    cout << "   ¿Desea procesar todo el proceso para otro curso? (S/N): ";
    cin >> repetir;

    repetir = toupper(repetir);

}while(repetir == 'S');

cout << "\nSistema finalizo con exito. Presione una tecla para salir..." << endl;
return 0;
```

7) 3 casos de prueba explicados

```
"C:\Users\User\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\CodeBlocks\ese.exe"
==========================================================================================


                RESUMEN ESTADISTICO DEL CURSO

1. Total de estudiantes procesados: 2
2. Estudiantes por paralelo: [A: 2] [B: 0] [C: 0]
3. Categorias: [Exc: 2] [MB: 0] [B: 0] [Riesgo: 0]
4. Cantidad de reprobados por nota: 0
5. Cantidad de reprobados por asistencia: 1
6. Promedio general del curso: 10.00
7. Promedio por paralelo:
   - Paralelo A: 10.00
   - Paralelo B: 0.00
   - Paralelo C: 0.00
8. Mayor promedio: 10.00 (Estudiante: SamuelBravo)
9. Menor promedio: 10.00 (Estudiante: SamuelBravo)
10. Porcentaje de aprobados: 50.00%
11. Porcentaje rendimiento estable: 100.00%
12. Porcentaje rendimiento inestable: 0.00%
==============================================================

--------------------------------------------------------------
   ┐Desea procesar todo el proceso para otro curso? (S/N): n

Sistema finalizo con exito. Presione una tecla para salir...

Process returned 0 (0x0)   execution time : 65.779 s
Press any key to continue.
```

Ilustración 1. Primera prueba de solución

```
"C:\Users\User\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\CodeBlocks\ese.exe"
==========================================================================================


                RESUMEN ESTADISTICO DEL CURSO

1. Total de estudiantes procesados: 1
2. Estudiantes por paralelo: [A: 0] [B: 0] [C: 1]
3. Categorias: [Exc: 0] [MB: 0] [B: 1] [Riesgo: 0]
4. Cantidad de reprobados por nota: 0
5. Cantidad de reprobados por asistencia: 0
6. Promedio general del curso: 7.10
7. Promedio por paralelo:
   - Paralelo A: 0.00
   - Paralelo B: 0.00
   - Paralelo C: 7.10
8. Mayor promedio: 7.10 (Estudiante: Sara Lopez)
9. Menor promedio: 7.10 (Estudiante: Sara Lopez)
10. Porcentaje de aprobados: 100.00%
11. Porcentaje rendimiento estable: 0.00%
12. Porcentaje rendimiento inestable: 0.00%
==============================================================

--------------------------------------------------------------
   ┐Desea procesar todo el proceso para otro curso? (S/N): n

Sistema finalizo con exito. Presione una tecla para salir...

Process returned 0 (0x0)   execution time : 27.434 s
Press any key to continue.
```

Ilustración 2. Segunda prueba de solución

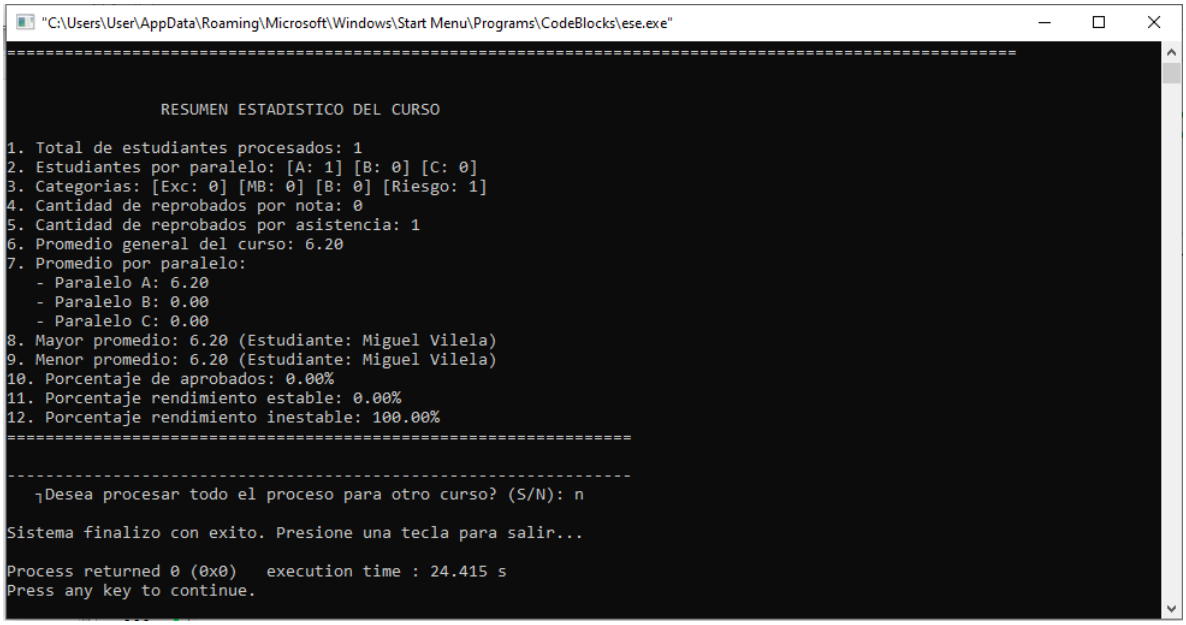


Ilustración 3.Tercera prueba d